1.体育课的铃声响了，同学们都陆续地奔向操场，按老师的要求从高到矮站成一排。每个同学按顺序来到操场时，都从排尾走到排头，找到第一个比自己高的同学，并站在他的后面。这种站队的方法类似于（ ）算法。

A. 快速排序

B. 插入排序

C. 冒泡排序

D. 归并排序

2.使用冒泡排序对序列进行升序排列，每执行一次交换操作系统将会减少 1 个逆序对，因此序列 5，4，3，2，1 需要执行（ ）次操作，才能完成冒泡排序。

A. 0

B. 5

C. 10

D. 15

3.

( )的平均时间复杂度为 $O(nlogn)$，其中 n 是待排序的元素个数。

A. 快速排序
B. 插入排序
C. 冒泡排序
D. 基数排序

4.

对于给定的序列$\{a_k\}$，我们把 (i,j) 称为逆序对当且仅当$i<j$且 $a_i>a_j$。那么 序列 {1, 7, 2, 3, 5, 4 } 的逆序对数为 ( ) 个。

A. 4
B. 5
C. 6
D. 7

5.

设 A 和 B 是两个长为 n 的有序数组，现在需要将 A 和 B 合并成一个排好序的数组，任何以元素比较作为基本运算的归并算法在最坏情况下至少要做（ ）次比较。

A. n^2

B. $nlogn$

C. $2n$

D. $2n-1$

6.

以下排序算法中，不需要进行关键字比较操作的算法是（ ）。

A. 基数排序

B. 冒泡排序

C. 堆排序

D. 直接插入排序

7.

（计数排序）计数排序是一个广泛使用的排序方法。下面的程序使用双关键字计数排序，将n对10000以内的整数，从小到大排序。

例如有三对整数(3,4)、(2,4)、(3,3),那么排序之后应该是(2,4)、(3,3)、(3,4)。

输入第一行为n,接下来n行，第i行有两个数$a[i]$和$b[i]$,分别表示第i对整数的第一关键字和第二关键字。

从小到大排序后输出。

数据范围 $1<n<10^7, 1<a[i], b[i]<10^4$。提示：应先对第二关键字排序，再对第一关键字排序。数组$ord[]$存储第二关键字排序的结果，数组$res[]$存储双关键字排序的结果。

试补全程序。

```
1       #include <cstdio>
2       #include <cstring>
3       using namespace std;
4       const int maxn = 10000000;
5       const int maxs = 10000;
6
7       int n;
8       unsigned a[maxn], b[maxn],res[maxn], ord[maxn];
9       unsigned cnt[maxs + 1];
10      int main() {
11              scanf("%d", &n);
12              for (int i = 0; i < n; ++i)
```

```
13                      scanf("%d%d", &a[i], &b[i]);
14              memset(cnt, 0, sizeof(cnt));
15              for (int i = 0; i < n; ++i)
16                      ①; // 利用 cnt 数组统计数量
17              for (int i = 0; i < maxs; ++i)
18                      cnt[i + 1] += cnt[i];
19              for (int i = 0; i < n; ++i)
20                      ②; // 记录初步排序结果
21              memset(cnt, 0, sizeof(cnt));
22              for (int i = 0; i < n; ++i)
23                      ③; // 利用 cnt 数组统计数量
24              for (int i = 0; i < maxs; ++i)
25                      cnt[i + 1] += cnt[i];
26              for (int i = n - 1; i >= 0; --i)
27                      ④ // 记录最终排序结果
28              for (int i = 0; i < n; i++)
29                      printf("%d %d", ⑤);
30
31              return 0;
32      }
```

①处应填（）
②处应填（）
③处应填（）
④处应填（）
⑤处应填（）

1.

A. `++cnt [i]`

B. `++cnt[b[i]]`

C. `++cnt[a[i] * maxs + b[i]]`

D. `++cnt[a[i]]`

2.

A. `ord[--cnt[a[i]]] = i`

B. `ord[--cnt[b[i]]] = a[i]`

C. `ord[--cnt[a[i]]] = b[i]`

D. `ord[--cnt[b[i]]] = i`

3.

A. `++cnt[b[i]]`

B. `++cnt[a[i] * maxs + b[i]]`

C. `++cnt[a[i]]`

D. `++cnt [i]`

4.

A. `res[--cnt[a[ord[i]]]] = ord[i]`

B. `res[--cnt[b[ord[i]]]] = ord[i]`

C. `res[--cnt[b[i]]] = ord[i]`

D. `res[--cnt[a[i]]] = ord[i]`

5.

A. `a[i], b[i]`

B. `a[res[i]], b[res[i]]`

C. `a[ord[res[i]]], b[ord[res[i]]]`

D. `a[res[ord[i]]], b[res[ord[i]]]`

第一空(3分)：　　第二空(3分)：

第三空(3分)：　　第四空(3分)：

第五空(3分)：

8.

以下排序算法的常见实现中，哪个选项的说法是错误的：（ ）。

A 冒泡排序算法是稳定的

B 简单选择排序是稳定的

C 简单插入排序是稳定的

D 归并排序算法是稳定的